

자바 웹 개발환경과 서블릿/JSP

오늘의 강의 학습 요약

금일 강의는 자바 개발 플랫폼을 **Java SE(표준)**와 **Java EE/Jakarta EE(웹)** 관점에서 재정리하고, 웹 개발을 위한 핵심 스펙인 **Servlet/JSP**가 "구현체가 아닌 스펙(인터페이스)"이라는 점을 JDBC 사례와 연결해 설명하는 흐름으로 진행되었습니다. 또한 해당 스펙을 구현한 대표 벤더로서 **Apache Tomcat(컨테이너/서버)**을 소개하고, 톰캣 버전과 스펙(서블릿/JSP) 및 패키지 네임스페이스 변화(javax → jakarta)까지 함께 정리했습니다.

이후 실습 중심으로 Eclipse Enterprise 버전 설치/설정, JDK 21 연동, Tomcat 압축 설치 및 Eclipse 서버 연동, 배포 경로(webapps)와 WAR 구조, 컨텍스트(Context) 개념(논리 이름)과 물리 디렉터리 매핑, Dynamic Web Project 생성, HTML/JSP 실행과 URL 구성, 서블릿 생성/매핑(어노테이션, web.xml), 그리고 서블릿 라이프사이클(init/service/destroy) 및 요청당 request/response 객체 생성 원리까지 전체 웹 개발 기본 동작을 완성형으로 다루었습니다.

1. 자바 플랫폼과 톰캣 스펙

Java SE는 JVM/JDK 기반의 비웹 애플리케이션 문법 학습 중심이며, Java EE(Jakarta EE)는 웹 애플리케이션을 위한 Servlet/JSP 스펙(인터페이스)과 이를 구현한 벤더 구현체(예: Tomcat)가 필요합니다.

Java 개발 플랫폼은 크게 SE, EE(엔터프라이즈), ME로 구분됩니다. SE는 지금까지 사용한 표준 환경이며, JDK 설치를 통해 JVM, 컴파일러(`javac`), 실행기(`java`), 그리고 기본 API(클래스 라이브러리)를 제공받는 구조입니다.

SE 애플리케이션은 시작점이 **main 메소드 1개**로 고정됩니다. 클래스가 많더라도 엔트리 포인트는 한 곳이므로 실행 흐름이 “main에서 시작”되는 형태로 정리됩니다.

Java EE(현 Jakarta EE)는 기업용 웹 애플리케이션 스펙이며, 자바 진영이 제공하는 것은 ****구현체가 아니라 스펙(명세)****입니다. 강의에서는 “스펙은 인터페이스로 이해”하는 방식으로 JDBC를 예시로 연결했습니다. JDBC가 인터페이스(스펙)이고 MySQL/Oracle/MSSQL 등이 이를 구현한 드라이버(구현 클래스)를 제공하는 것처럼, Servlet/JSP도 동일한 구조입니다.

스펙을 실제로 실행하려면 구현체와 실행 환경이 필요하며, 그 대표가 **Apache Tomcat**입니다. 톰캣은 서블릿/JSP 구현체(JAR)와 웹 실행 환경을 함께 제공하므로 웹 애플리케이션 개발/테스트에 필수입니다.

톰캣 버전과 스펙/패키지 변화는 반드시 함께 확인해야 합니다. 톰캣 10 이후로 네임스페이스가 `javax.*` 에서 `jakarta.*` 로 전환되었으며, 이는 “스펙 관리 주체가 Oracle(과거 Sun) → Apache로 이관”된 배경과 연결됩니다. 또한 톰캣 문서에서 해당 버전이 지원하는 서블릿/JSP 스펙 버전을 확인하는 습관이 필요합니다.

구분	Tomcat 9	Tomcat 10/11 계열
관리 주체 관점	Oracle 중심 시기의 <code>javax</code> 네임스페이스가 주로 등장합니다.	Apache(Jakarta) 중심의 <code>jakarta</code> 네임스페이스를 사용합니다.
JDK 지원 개요 (강의 기준)	JDK 10까지로 설명되었습니다.	JDK 17 이상에서 사용 가능한 흐름으로 설명되었습니다.
패키지 네임스페이스	<code>javax.servlet.*</code> 계열이 등장합니다.	<code>jakarta.servlet.*</code> 계열이 등장합니다.
스펙 확인 방법	Tomcat Documentation에서 지원 스펙을 확인해야 합니다.	Tomcat Documentation에서 지원 스펙을 확인해야 합니다.

핵심 키워드

#Java SE

#Jakarta EE

#Servlet/JSP 스펙

#Tomcat

#javax→jakarta

2. 개발환경 구축: JDK, 이클립스, 톰캣

웹 개발환경은 JDK(예: 21) 설치 및 환경변수 설정, Eclipse Enterprise 패키지 선택, Tomcat ZIP 설치 후 Eclipse 서버 연동까지가 하나의 세트로 구성됩니다.

엔터프라이즈 환경을 학습하더라도 JDK는 필수입니다. 강의에서는 JDK 21 설치를 전제로 `JAVA_HOME` 과 `PATH` 설정을 재확인했습니다. `PATH`에는 보통 `bin`까지 포함되며, 환경변수 치환은 `%JAVA_HOME%` 형태로 사용합니다.

아래는 강의에서 강조된 Windows 환경변수 구성의 핵심 형태입니다.

```
JAVA_HOME=C:\Program Files\Java\jdk-21
PATH=%JAVA_HOME%\bin;%PATH%
```

Eclipse는 “무엇을 개발하느냐”에 따라 배포판이 달라집니다. 웹 개발을 위해서는 **Eclipse IDE for Enterprise Java** 계열이 필요하며, 단순 Java(SE) 용 패키지로는 웹 프로젝트 구성이 제한됩니다. 또한 최신 Eclipse가 항상 정답은 아니며, **JDK 지원 범위** 때문에 특정 버전(강의에서는 2025 계열)을 선택하는 이유를 설명했습니다.

Tomcat은 설치형 실행 파일(exe)보다 ZIP 압축본을 내려받아, Eclipse와 연동하여 사용하는 방식을 채택했습니다. 이 경우 “압축을 어디에 풀었는지”가 경로 설정의 기준이 되므로, 강의에서는 **현재 폴더에 풀기** 방식으로 경로를 통일했습니다.

Eclipse 실행 후에는 설정을 반드시 점검합니다. 특히 웹 개발에서는 인코딩과 컴파일러 버전, 설치된 JRE 연결이 빌드/실행 오류를 좌우하므로, 아래 항목을 확인하도록 안내했습니다.

설정 항목	위치(개념)	기대 값(강의 기준)
워크스페이스 인코딩	Preferences > General > Workspace	UTF-8
Java 컴파일러 버전	Preferences > Java > Compiler	JDK 21 사용 기준에 맞게 설정합니다.
Installed JRE	Preferences > Java > Installed JREs	Eclipse 내장 JRE가 아니라 설치한 JDK를 사용합니다.
웹 리소스 인코딩	Preferences > Web > CSS/HTML/JS	UTF-8
<code>.properties</code> 인코딩	Preferences > General > Content Types > Text > Java Properties File	UTF-8 후 Update 적용합니다.

Tomcat을 단독으로 실행할 경우 `startup.bat / shutdown.bat` 를 사용할 수 있지만, 독립 실행에는 `CATALINA_HOME` 환경변수가 필요할 수 있습니다. 다만 강의에서는 Eclipse 연동 방식이므로 `CATALINA_HOME` 을 필수로 두지 않았습니다.

핵심 키워드

`#JAVA_HOME`

`#Eclipse Enterprise`

`#Tomcat ZIP`

`#UTF-8`

`#CATALINA_HOME`

3. 이클립스-톰캣 연동과 서버 설정

Eclipse의 Servers 뷰에서 Tomcat 10.1 런타임을 추가하고, Server Location과 Deploy Path(webapps)를 올바르게 지정한 뒤 8080으로 접속해 톰캣 홈이 출력되면 연동이 완료됩니다.

Eclipse의 **Servers** 뷰에서 새 서버를 추가하여 Tomcat을 연동합니다. 이때 버전 선택이 중요하며, 강의에서는 다운로드한 Tomcat과 일치하는 **Tomcat 10.1** 런타임을 선택하도록 강조했습니다.

서버 생성 과정에서 Tomcat Home 디렉터리는 ****bin 이전(톰캣 루트)****까지 지정해야 합니다. 또한 JRE는 기본 값을 쓰더라도, 강의에서는 명시적으로 설치한 JDK를 선택하는 흐름을 보여주었습니다.

서버를 생성하면 끝이 아니라, 생성된 서버를 더블 클릭해 Overview에서 세부 설정을 마무리해야 합니다. 특히 다음 두 설정은 강의에서 “가장 중요하며, 한 번 저장하면 수정이 어렵다”는 점을 명확히 했습니다.

- Server Location은 “Use Tomcat installation” 계열로 설정되어야 합니다.
- Deploy Path는 Tomcat의 `webapps` 디렉터리로 맞춰져야 합니다.
- 포트는 기본 8080을 사용하며, 필요 시 변경 가능하다고 설명했습니다.
- 저장 후 대부분 설정이 비활성화되어, 오설정 시 삭제 후 재등록이 필요합니다.

삭제가 필요한 경우, “Servers 뷰에서만 지우면 끝”이 아니라, Preferences의 Server Runtime 환경 등에서도 함께 제거해야 깔끔하게 정리됩니다. 강의에서는 삭제 대상이 3군데에 남을 수 있음을 짚었습니다.

서버 실행은 Servers 뷰의 **Start(녹색)** 버튼으로 수행하며, 종료는 **Stop(빨간색)** 버튼으로 수행합니다. 콘솔에 빨간 로그가 출력되더라도 INFO 로그일 수 있으므로 “빨간색=에러”로 단정하지 말라고 안내했습니다.

마지막으로 브라우저에서 `http://localhost:8080/` 접속 시 톰캣 기본 홈 페이지가 출력되면 서버 구동이 성공입니다.

핵심 키워드

#Servers 뷰

#Tomcat 10.1 런타임

#webapps

#8080

#서버 재등록

4. 배포 구조, 컨텍스트, URL 해석

톰캣의 webapps는 배포 대상(웹 애플리케이션)의 물리 디렉터리이며, 브라우저 URL의 컨텍스트 경로(논리 이름)가 물리 디렉터리와 매핑되어 리소스를 찾는 구조입니다.

웹 애플리케이션 배포의 기본 원리는 “WAR로 묶어 webapps 에 배치하면 톰캣이 자동으로 풀어준다”는 구조입니다. WAR 내부는 WEB-INF 가 핵심이며, WEB-INF/classes 에는 컴파일된 클래스가 들어가고 web.xml 이 설정 파일로 존재합니다.

강의에서는 webapps에 기본 배포된 예시(예: examples , manager , ROOT)를 직접 확인하면서, 각 웹 앱이 동일한 구조를 갖는다는 점을 반복적으로 강조했습니다.

구성 요소	의미	접근 특성
webapps/<앱>	웹 애플리케이션 물리 디렉터리입니다.	PC 파일시스템에 남는 물리 자원입니다.
WEB-INF/	배포 메타/보안 영역입니다.	브라우저에서 직접 접근이 불가능합니다.
WEB-INF/classes/	컴파일된 .class 가 위치합니다.	런타임 클래스 로딩 대상입니다.
WEB-INF/lib/	외부 라이브러리(JAR)를 둡니다.	넣기만 하면 클래스패스로 인식됩니다.

SE 환경에서는 외부 라이브러리를 Build Path로 직접 추가했지만, 웹 환경에서는 WEB-INF/lib 에 JAR를 넣는 것이 표준적인 방식이며, 그 자체가 빌드패스 효과를 냅니다.

URL 해석의 핵심은 **컨텍스트(Context)**입니다. webapps 의 물리 디렉터리 이름과 브라우저에서 사용하는 논리 이름(컨텍스트 경로)이 매핑되어 있으며, 브라우저는 컨텍스트 경로를 통해 톰캣이 어느 물리 디렉터리를 찾아갈지 결정합니다.

루트 앱(ROOT)은 예외적으로 컨텍스트가 / 로 매핑되며, 따라서 URL에서 루트는 생략되는 것처럼 보입니다. 반면 다른 앱(examples, manager 등)은 반드시 컨텍스트를 명시해야 접근됩니다.

기본 페이지가 파일명을 쓰지 않아도 열리는 이유는 “Welcome File List” 설정 때문입니다. 서버 전역 web.xml 의 welcome-file-list 에 index.html , index.jsp 등이 등록되어 있으면, 파일명을 생략해도 순서대로 탐색해 기본 페이지를 선택합니다.

핵심 키워드

#webapps

#WAR

#WEB-INF

#컨텍스트 경로

#welcome-file-list

5. Dynamic Web Project와 리소스 배치

Dynamic Web Project는 SRC(main/java)와 WebContent(웹 리소스)로 역할이 분리되며, 컨텍스트 루트가 URL의 논리 경로가 되고 생성 시 web.xml(Deployment Descriptor) 생성 여부를 결정합니다.

프로젝트는 **File > New > Dynamic Web Project**로 생성합니다. 생성 시 Target Runtime이 Tomcat으로 선택되어야 하며, 그렇지 않으면 서버 실행/배포가 정상 동작하지 않을 수 있습니다.

프로젝트 구조에서 Java 소스는 `src/main/java`에 저장되고, HTML/CSS/JS/JSP 등 웹 리소스는 Web 계층(강의에서 WebContent/Webapp로 지칭된 영역)에 저장됩니다. 이는 SE에서 `src` 하나로 끝났던 구조와 다르므로 경로를 정확히 구분해야 합니다.

또한 프로젝트 생성 화면의 **Context Root**는 브라우저 URL의 컨텍스트(논리 이름)이며, 기본적으로 프로젝트명과 같게 잡히지만 변경할 수 있습니다. 이 값이 `http://localhost:8080/<context>/...`의 `<context>`가 됩니다.

`Generate web.xml deployment descriptor`를 체크하면 프로젝트별 `web.xml`이 생성됩니다. 강의에서는 `web.xml`을 ****Deployment Descriptor(배치 지시자)****라고 부르며, 웹 애플리케이션 단위 설정 파일이라는 의미를 강조했습니다.

Eclipse와 Tomcat을 연동해두면, 프로젝트 생성 및 실행 시 WAR 수동 배포 없이도 **자동 배포**가 이루어집니다. 즉, Eclipse가 톱캣의 `webapps`로 필요한 구조를 맞춰 배치해주므로, 학습 초반에는 배포 작업 부담이 크게 줄어듭니다.

핵심 키워드

#Dynamic Web Project

#Context Root

#src/main/java

#Deployment Descriptor

#자동 배포

6. HTML, JSP 실행과 URL 구성

HTML과 JSP는 웹 리소스로서 WEB-INF 밖에 두어야 하며, 실행 시 URL은 로컬호스트/포트/컨텍스트/폴더/파일 순으로 구성됩니다.

HTML은 웹 리소스이므로 Web 영역(웹 루트)에 만들어야 하며 WEB-INF 내부에 두면 브라우저에서 직접 접근할 수 없습니다. 이는 WEB-INF 가 보안 영역이기 때문이며, 브라우저 접근 대상은 웹 루트(=WEB-INF와 같은 레벨)입니다.

폴더를 만들면 URL에도 동일한 경로가 포함됩니다. 예를 들어 컨텍스트가 hello 이고, my/main2.html 을 만들면 URL은 /hello/my/main2.html 형태가 됩니다.

실행은 가장 간단히 파일 우클릭 후 **Run As > Run on Server**로 수행합니다. 이 과정에서 서버가 필요하면 자동으로 구동/재시작되며, 브라우저 요청 URL이 자동 구성되므로 URL 패턴을 함께 관찰하는 것이 핵심입니다.

JSP는 HTML과 유사한 위치에 저장하며, 실행/요청 방식도 동일합니다. 다만 JSP는 동적 컴포넌트로서 서버 측에서 처리된 결과가 HTML로 렌더링된다는 점이 다릅니다.

JSP 실행은 내부적으로 3단계를 거칩니다.

단계	설명
변환 단계	hello.jsp 가 hello_jsp.java 형태로 변환됩니다.
컴파일 단계	변환된 Java가 .class 로 컴파일됩니다.
실행 단계	실행 결과로 HTML이 생성되어 브라우저에 렌더링됩니다.

이 변환/컴파일 산출물은 톰캣의 work 디렉터리에서 확인할 수 있으며, JSP 오류가 발생하면 "JSP 자체"뿐 아니라 변환된 Java/컴파일 과정에서의 오류 가능성까지 고려해야 한다는 점을 강조했습니다.

핵심 키워드

#WEB-INF 접근 제한

#Run on Server

#URL 경로

#JSP 변환/컴파일

#Tomcat work

7. 서블릿 개념, 생성 규칙, 매핑

서블릿은 main이 없는 동적 컴포넌트이며, 컨테이너가 생성, 소멸을 관리하고 URL 매핑(별칭)을 통해 안전하고 짧은 경로로 호출합니다.

서블릿은 Java 코드이므로 `src/main/java` 에 저장되며, 웹 환경에서는 패키지 사용이 사실상 필수로 안내되었습니다. 또한 서블릿은 main 메소드가 없는데, 이는 웹 요청이 “사용자가 어떤 URL을 먼저 치느냐”에 따라 시작점이 달라지기 때문입니다.

서블릿 객체는 개발자가 `new` 로 생성하지 않습니다. 톰캣 같은 컨테이너가 서블릿의 생성부터 소멸까지(라이프사이클) 관리합니다. 이 점 때문에 강의에서는 “서버”보다 “컨테이너”라는 용어가 더 본질에 가깝다고 설명했습니다.

서블릿 구현의 핵심 규칙은 다음과 같이 정리됩니다.

- 서블릿은 패키지로 구성되어야 합니다.
- `jakarta.servlet.http.HttpServlet` 을 상속해야 합니다.
- 요청을 처리하려면 `doGet()` 또는 `doPost()` 를 재정의해야 합니다.
- 브라우저가 접근할 수 있도록 URL 매핑이 필요합니다.

서블릿 계층 구조도 함께 정리했습니다. `HttpServlet` 은 추상 클래스이며 상위에 `GenericServlet` (추상 클래스), 최상위에 `Servlet` (인터페이스)이 존재합니다. 또한 `GenericServlet` 은 `Servlet` 과 `ServletConfig` 를 구현한다는 점을 API 문서로 확인했습니다.

서블릿 매핑은 “과거 방식(긴 URL)”을 대체하기 위한 필수 개념입니다. 예전에는 `/servlet/<패키지 포함 클래스명>` 처럼 노출이 많고 길었으나, 보안과 사용성 문제가 커서 현재는 매핑(별칭)을 사용합니다.

매핑 방법은 2가지입니다.

방식	설정 위치	핵심 포인트
어노테이션	<code>@WebServlet("/test")</code>	슬래시는 필수이며 코드에서 바로 관리합니다.
web.xml	<code><servlet></code> , <code><servlet-mapping></code>	<code>servlet-name</code> 으로 선언/매핑을 연결하고 URL 패턴은 유일해야 합니다.

web.xml 방식에서는 태그가 스키마(XSD)에 의해 제한되며, `servlet-name` 은 `<servlet>` 과 `<servlet-mapping>` 을 연결하는 키 역할을 합니다. 또한 `url-pattern` 은 애플리케이션 내에서 중복되면 어떤 서블릿으로 갈지 결정할 수 없어 오류가 발생합니다.

핵심 키워드

#HttpServletRequest

#@WebServlet

#web.xml 매핑

#ServletConfig

#URL pattern

8. 서블릿 라이프사이클과 요청 객체 설계

`init/destroy`는 한 번만 호출되고 `doGet/doPost`는 요청마다 호출되며, `request/response`는 요청당 로컬로 생성되어 사용자 간 데이터가 섞이지 않도록 설계됩니다.

서블릿 라이프사이클의 콜백 메소드는 컨테이너가 호출합니다. `init()` 은 서블릿 생성 시 1회, `destroy()` 는 소멸 시 1회 호출되며, `doGet()` / `doPost()` 는 요청이 들어올 때마다 매번 호출됩니다.

중요한 오해를 바로잡는 부분이 핵심입니다. 여러 사용자가 접근한다고 해서 서블릿 인스턴스가 매번 새로 생성되는 것이 아니라, 서블릿은 보통 **한 번 생성되어 재사용**됩니다. 동시 요청은 내부적으로 스레드 처리로 확장되는 방식으로 설명되었습니다.

요청을 처리하는 `doGet/doPost` 는 다음 두 파라미터를 받으며, 이것이 “로컬 변수(파라미터)”라는 점을 매우 강하게 강조했습니다.

- `HttpServletRequest request` 는 요청 데이터를 다루는 API입니다.
- `HttpServletResponse response` 는 응답을 구성하는 API입니다.
- 이 객체들은 요청당 생성되고 응답 후 소멸되어야 합니다.
- 이를 통해 사용자 A/B/C의 요청 데이터가 서로 섞이지 않습니다.

반대로 서블릿이 단일 인스턴스로 재사용되는 특성 때문에, **인스턴스 변수**를 선언하면 여러 사용자가 공유하는 상태가 될 수 있습니다. 이는 보안/정합성 문제를 만들 수 있으므로, 사용자 요청 단위 데이터는 반드시 메소드 내부의 **로컬 변수**로 내려서 처리해야 한다는 결론을 제시했습니다.

실습에서는 `init()` 과 `destroy()` 에 로그를 찍어, 최초 요청 시 `init` 이 1회만 출력되고 이후 새로고침/다른 브라우저 요청에서는 `doGet` 만 반복 호출되는 것을 확인했습니다. 또한 코드 수정 등으로 서버가 리스타트될 때 `destroy()` 가 호출되는 흐름도 관찰했습니다.

핵심 키워드

`#init()`

`#doGet/doPost`

`#destroy()`

`#HttpServletRequest`

`#인스턴스 변수 공유 위험`